Project Architecture & Execution Plan

Natural Language to SQL (NL2SQL) for JanAadhaar Database

1. Project Overview

This document outlines the architectural approach and execution strategy for developing a Natural Language to SQL (NL2SQL) retrieval module. The system is designed to interface with the Rajasthan Government's JanAadhaar database, which houses approximately 8 crore (80 million) records across 60-100 columns. The primary objective is to allow users to query this massive dataset using plain English/Natural Language, which the system will dynamically translate into optimized, secure, and accurate SQL queries.

2. Core Metrics

- The success of this module is driven by two primary metrics:
- Efficiency: The system must scale gracefully to handle 8 crore entries, minimizing query latency and LLM token overhead.
- Accuracy: The generated SQL must perfectly reflect the user's intent without hallucinating table names or columns, ensuring deterministic data retrieval.

3. Technology Stack

Backend Framework: Python with FastAPI for high-performance API endpoints.
Database: MySQL (or similar SQL variant) optimized with rigorous indexing/hashing.
LLM / AI Model: External LLM API for intent extraction and SQL generation.
Vector Database: Qdrant or ChromaDB for RAG-based schema retrieval.
Validation Layer: Custom SQL validation middleware to restrict operations.

4. Architectural Workflow (RAG Approach)

To prevent context window overflow and reduce hallucinations, we will not feed the entire 60-100 column schema to the AI directly. Instead, we are implementing a Retrieval-Augmented Generation (RAG) pipeline:

1. Step 1: Intent & Entity Extraction: The natural language prompt is parsed to identify key intents and entities (e.g., 'top-selling laptops in Delhi last month' -> intent: MAX(sales), product: laptop, location: Delhi).
2. Step 2: Vector Search (Schema Retrieval): We query the Vector DB (ChromaDB/Qdrant) using the extracted entities. The Vector DB contains embeddings of all table

descriptions, column headers, and mapped synonyms. It acts like a 'Google Search for Schema', returning only the relevant column definitions.

3. Step 3: SQL Generation: The relevant subset of the schema, along with the user's query, is sent to the LLM to generate the SQL structure.
4. Step 4: Parameterization (Security): To prevent SQL injection, the LLM is instructed to return a JSON object separating the query structure from the values. Expected output format: {"sql": "SELECT ... WHERE col = ?", "paramValues": [value]}.
5. Step 5: Validation & Execution: The generated SQL passes through a strict validator (allowing ONLY 'SELECT' queries). If valid, it is executed against the database.
6. Step 6: Human-Friendly Output (Optional): The fetched data is passed back through a lightweight LLM prompt to convert the raw tabular data into a natural language summary.

5. Data Preparation & Schema Optimization

For the RAG model to successfully map natural language to the correct columns, data preparation is critical:

- Clear Headings & Descriptions: Every column will be documented with a clear, descriptive text string indicating its purpose.
- Synonym/Alias Mapping: We will hardcode specific aliases to map to entities to save processing time and improve vector search accuracy (e.g., Table 'Products' maps to aliases: items, goods, phones, electronics).
- Clean Data Structures: Ensuring the underlying MySQL tables are normalized where necessary, though heavily indexed for read-heavy operations.

6. Anticipated Challenges & Mitigations

- Challenge: Database Scalability (8 Crore Records)

  Mitigation: Implementing advanced indexing and hashing strategies on frequently queried columns (e.g., location, dates, IDs).

- Challenge: SQL Injection Attacks

  Mitigation: Strict abstraction of user inputs. The AI will generate parameterized queries, isolating string/value injection risks. Furthermore, database credentials will be strictly limited to Read-Only access.

- Challenge: Missing Parameters or Ambiguous Queries

  Mitigation: Implementing a feedback loop in the API. If the LLM identifies missing crucial information, it returns a prompt for clarification rather than executing an overly broad query.

- Challenge: Malformed SQL Queries

  Mitigation: SQL validation middleware will catch syntax errors or non-SELECT operations before hitting the database, triggering an automatic self-correction prompt back to the LLM.

7. Caching Strategy (Look-Aside Architecture)

To further optimize efficiency, reduce database load, and minimize LLM API token costs, a multi-tier caching strategy will be implemented utilizing a Look-Aside (Cache-Aside) buffer approach. An in-memory data store (such as Redis or Memcached) will serve as the caching layer.

- Frequent Queries (Semantic & SQL Caching): Before processing a natural language prompt through the LLM, the system will check if an identical or highly similar query has been requested recently. If found, the cached SQL structure is retrieved instantly, bypassing the LLM extraction phase.
- Frequently Retrieved Data: The actual dataset results of highly frequent or computationally heavy SQL queries (e.g., top-level aggregates or common demographic statistics) will be temporarily stored in the cache.
- Time-to-Live (TTL) Limits: To maintain data accuracy and prevent the serving of stale information, all cached data will have a strict, limited-time TTL. Once the TTL expires, the cached entry is automatically invalidated.
- Look-Aside Buffer Workflow: The application code will first check the cache for the requested data (Cache Hit). If the data is absent (Cache Miss), the application will execute the generated SQL against the primary MySQL database, return the payload to the user, and immediately populate the cache with this new data for subsequent requests.

8. Robust SQL Validation & Guardrails Layer

To achieve absolute accuracy and ironclad system stability, a dedicated SQL Validation and Guardrails layer sits between the LLM output and the MySQL execution engine. This layer handles syntactical correctness, hallucination filtering, and structural compliance before any query hits the 8-crore dataset.

- Syntactical and Structural Validation: The generated SQL string is parsed using a programmatic SQL parser (e.g., sqlglot or sqlparse) to catch syntax errors, missing commas, mismatched parentheses, or malformed clauses before execution, completely preventing runtime database crashes.
- Anti-Hallucination Schema Cross-Checking: The validator strictly checks all tables, aliases, and column names extracted from the LLM's JSON output against the master

schema catalog. If the LLM hallucinates a non-existent column or misinterprets a data type, the validator immediately flags the mismatch and blocks execution.

- Strict Read-Only Enforcement: The middleware inspects the Abstract Syntax Tree (AST) of the query. It explicitly blocks any operations containing forbidden SQL tokens such as INSERT, UPDATE, DELETE, DROP, ALTER, or GRANT, enforcing a strict whitelist that only permits safe SELECT data retrieval.
- Self-Correction Loop: If a query fails validation (due to syntax errors or hallucinated columns), the system triggers an automated internal retry mechanism. The validation error log is packaged into a corrective prompt and sent back to the LLM (e.g., 'Your query failed because column X does not exist. Please rewrite using the available schema...'), allowing the model to self-heal without user intervention.